

BEGINNER EMPLOYEE API

Employee and Department Management System

A Project Documentation

Presented to

The University of Mindanao

College of Computing Education

In Partial Fulfillment of the Requirements in CCE104

1st Semester, 1st Term S.Y. 2026-2027

RENZ R. EGONIA

October 2026

Executive Summary

The Beginner Employee API is a small information system for managing company departments, employee records, and department transfers. It is built with Python, FastAPI, SQLite, and Pydantic. The system provides CRUD endpoints for departments and employees, search and filters, a staffing summary report, and a transfer workflow that updates an employee and writes an audit record in one database transaction.

The application expands the supplied FastAPI boilerplate rather than submitting it unchanged. It keeps the straightforward `main.py` and `app/` organization and adds related tables, service classes, validation rules, transfer-history controls, and automated integration tests.

Contents

- 1. Problem identification and objectives
- 2. Scope, users, and technologies
- 3. Architecture and object-oriented design
- 4. Database design and ER diagram
- 5. Table structure and integrity constraints
- 6. API endpoint documentation
- 7. Employee-transfer transaction workflow
- 8. Installation and Swagger demonstration
- 9. Testing, limitations, and GitHub publication

1. Problem Identification and Objectives

Problem. Small offices may track employees and their department assignments in spreadsheets or paper records. When an employee moves between departments, staff may forget to update every copy of the record, duplicate entries may be created, and the reason for a transfer may not be retained. It can also be difficult to produce an accurate staffing summary quickly.

Proposed solution. This API maintains employee and department information in a single relational database, validates incoming requests, enforces foreign-key rules, and records department transfers with a reason and timestamp. The API can be operated through FastAPI Swagger UI without requiring a separate frontend.

Project objectives

- Centralize department and employee records in SQLite.
- Provide create, retrieve, search, update, and delete operations with input validation.
- Use primary and foreign keys to protect related data.
- Record employee department changes in an auditable, atomic workflow.
- Apply object-oriented organization through reusable service classes.
- Provide a staffing report and automated tests for important behaviors.

2. Scope, Users, and Technologies

Intended users

- Human Resources staff: maintain employee records and department assignments.
- Department heads: retrieve employees by department and review the staffing summary.
- Project evaluator or developer: test API behavior using Swagger UI and automated tests.

Included functions

- Department CRUD with unique department names.
- Employee CRUD with a required existing department and positive salary.
- Search by employee name or position and filtering by department or employment status.
- Department transfer with transfer reason and historical record.
- Summary report with employee totals, active counts, headcount, and average salary.
- Validation and consistent HTTP error handling for invalid or conflicting requests.

Out of scope

The prototype does not include user accounts, authentication, role-based access control, a graphical management dashboard, payroll processing, attendance management, or production hosting. It is intended for academic demonstration with sample data, not confidential employee data.

Technology stack

Technology	Purpose in the project
Python 3.10+	Application language and service logic.
FastAPI	HTTP routing, request handling, response serialization, and automatic OpenAPI documentation.
Pydantic v2	Validates request fields and constrains accepted data.
SQLite	Stores the three related tables in a local database file.
pytest + FastAPI TestClient	Runs integration tests against API endpoints using an isolated test database.
Uvicorn	Starts the local ASGI server used to run the API.

No external database server is required. On first startup, the application creates the SQLite database and the required tables automatically.

3. Architecture and Object-Oriented Design

The project separates the HTTP layer, validated request/response models, database setup, and business rules. API route functions in `main.py` handle the request and call service objects. The services execute parameterized SQL statements and apply rules before changing the database.

File or class	Responsibility
main.py	Creates the FastAPI application, registers endpoints, defines error handlers, and connects routes to service objects.
app/database.py	Defines the database path, connection context manager, foreign-key enforcement, commit/rollback behavior, tables, and indexes.

File or class	Responsibility
app/models.py	Defines Pydantic input/output models, validates field lengths and allowed employment statuses, and documents response schemas.
app/services.py	Implements database operations and business rules inside DepartmentService and EmployeeService.
DepartmentService	Encapsulates department create/read/update/delete rules and handles duplicate names or protected deletion.
EmployeeService	Encapsulates employee CRUD, filtering, department transfers, transfer-history retrieval, and summary calculations.
tests/test_api.py	Tests complete HTTP requests and the resulting database behavior.

OOP concepts demonstrated

- Classes and objects: `DepartmentService` and `EmployeeService` are classes; `department_service` and `employee_service` are application objects used by route functions.
- Encapsulation: database operations and rules are grouped in services instead of being repeated in every endpoint.
- Separation of responsibilities: Pydantic models handle validation, services handle use cases, database.py manages transactions, and main.py handles HTTP.
- Reuse: common behavior, such as opening a connection and turning rows into dictionaries, is shared by multiple operations.

4. Database Design and ER Diagram

The database contains three tables. `departments` is the parent table for employee department assignment. `employees` references one department. `department_transfers` references one employee and two department rows: the previous department and the destination department.

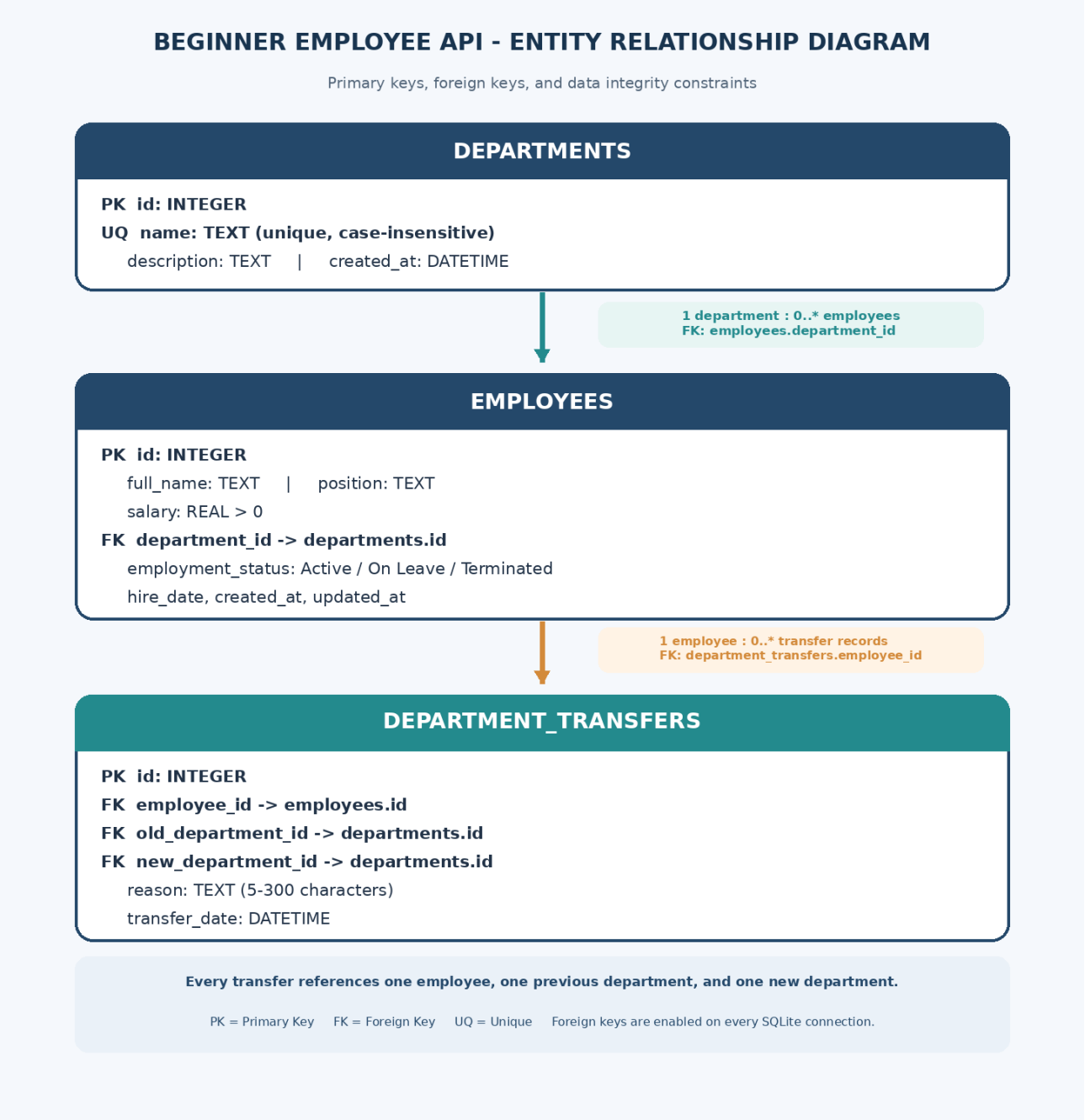


Figure 1. Entity relationship diagram for the Employee API database.

Relationship and participation rules

- Department to Employee: one department can have zero or many employees; each employee must reference exactly one existing department.
- Employee to DepartmentTransfer: one employee can have zero or many transfer records; each transfer belongs to exactly one employee.
- Department to DepartmentTransfer: a department can be referenced by many transfer records as an old or new department; each transfer has exactly one old and one new department. The IDs must differ, and the reason is retained for auditing.

5. Table Structure and Integrity Constraints

departments

Column	Rule / meaning
id	Integer primary key; automatically generated.
name	Required, 2-80 characters, unique without case sensitivity.
description	Optional description; defaults to an empty string.
created_at	Timestamp generated by SQLite.

employees

Column	Rule / meaning
id	Integer primary key; automatically generated.
full_name / position	Required text with length limits and whitespace validation.
salary	Numeric monthly salary greater than zero.
department_id	Required foreign key referencing departments.id; deletion is restricted.
employment_status	Must be Active, On Leave, or Terminated.
hire_date / timestamps	Hire date, creation time, and last-update time.

department_transfers

Column	Rule / meaning
id	Integer primary key.
employee_id	Foreign key referencing employees.id.
old_department_id	Foreign key referencing the department before the transfer.
new_department_id	Foreign key referencing the destination department.
reason	Required, 5-300 characters.
transfer_date	Timestamp created by SQLite.

Additional database safeguards include `NOT NULL`, `UNIQUE`, `CHECK`, foreign-key restrictions, and indexes for common department/name/history lookups. SQLite foreign-key enforcement is enabled every time a connection opens. The matching SQL is also included in `docs/schema.sql`.

6. API Endpoint Documentation

The API can be tested at `http://127.0.0.1:8000/docs` after the server is running. The local address works on the computer running the server; it is not a public website.

Department endpoints

Method and path	Purpose	Typical result
GET /departments	List departments.	200 OK
GET /departments/{department_id}	Retrieve one department by ID.	200 / 404
POST /department	Create a department.	201 / 409 / 422
PUT /department/{department_id}	Update department details.	200 / 404 / 409 / 422
DELETE /department/{department_id}	Delete only when no records reference it.	204 / 404 / 409

Employee endpoints

Method and path	Purpose	Typical result
GET /employees	List; optional search, department_id, status filters.	200 OK
GET /employees/{employee_id}	Retrieve one employee.	200 / 404
POST /employee	Create an employee in an existing department.	201 / 404 / 422
PUT /employee/{employee_id}	Update details without changing departments.	200 / 404 / 409 / 422
DELETE /employee/{employee_id}	Delete only if no transfer history exists.	204 / 404 / 409

Workflow, report, and status codes

Method and path	Purpose
POST /employees/{employee_id}/transfer	Change department and insert a transfer-history row in one transaction.
GET /employees/{employee_id}/transfers	Retrieve historical transfers for an employee.
GET /reports/summary	Return totals and per-department headcount, active count, and average salary.
GET /	Return a welcome message and documentation path.

Common status codes: 200 means a request succeeded; 201 means a record was created; 204 means a delete succeeded with no response body; 404 means the requested record does not exist; 409 means a uniqueness or relationship rule blocks the operation; and 422 means the request failed input validation.

7. Employee Transfer Transaction Workflow

The transfer process is the system's main transactional workflow. The API validates the request, checks the employee and destination department, updates the employee's department, inserts a history record, and commits both writes only after they succeed. The connection context manager rolls back the transaction if an exception occurs.



Figure 2. Transactional employee department transfer workflow.

Why the transaction matters

Without a transaction, the employee row might be moved to a new department even if the history insert fails. The code wraps both SQL writes in the same connection context. A deliberate test trigger forces the history insert to fail and verifies that the employee remains in the original department and no partial history is created.

8. Installation and Swagger Demonstration

Windows (PowerShell)

Extract the project ZIP, open a terminal inside the `Employee_API_Final` folder, and run:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m uvicorn main:app --reload
```

If activation is blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` in that same PowerShell window, then activate the environment again.

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m uvicorn main:app --reload
```

Suggested live demonstration in /docs

- Create two departments using POST /department and copy the IDs from the responses.
- Create an employee using POST /employee and an existing department_id.
- Retrieve the employee through GET /employees and GET /employees/{employee_id}.
- Use the search/filter parameters on GET /employees.
- Transfer the employee to the second department through POST /employees/{employee_id}/transfer.
- Verify the new department through GET /employees/{employee_id}; then retrieve GET /employees/{employee_id}/transfers.
- Open GET /reports/summary to show employee totals and department-level statistics.
- Try invalid salary, an unknown department ID, or a repeated department name to demonstrate validation and errors.

For submission evidence, capture screenshots from this updated application after running it locally: the Swagger endpoint list, a successful department/employee creation, the transfer response and history, and the summary report. Do not use an old screenshot that lacks the added endpoints.

9. Testing, Limitations, and GitHub Publication

Automated test results

The integration suite was executed against an isolated temporary SQLite database. The latest run completed with 10 passing tests.

Test area	What is verified
Startup and retrieval	Welcome route and initially empty department/employee lists.
CRUD	Department and employee creation, retrieval, update, and deletion.
Validation	Duplicate department names, invalid salary, short transfer reason, and missing departments.
Search and filters	Employee text search and filters by department and employment status.
Not-found handling	Unknown departments/employees and missing transfer-history requests return 404.
Transfer workflow	Transfer changes the assigned department and adds an auditable history record.
Relationship protection	Same-department transfers, unsafe deletes, and direct unrecorded department changes are rejected.
Transaction rollback	A simulated transfer-history insert failure leaves the employee in the original department.

```
python -m pytest -v
Expected summary: 10 passed
```

Limitations and future improvements

- Add authentication and role-based permissions before any shared deployment.
- Create a web frontend for staff who should not work directly in Swagger UI.
- Add backup/export tools and deployment configuration if the system grows beyond a classroom prototype.
- Keep the database file and real employee information out of public repositories.

GitHub publication steps

The project source is prepared as a separate package based on the supplied boilerplate. To complete the submission, create a new repository under your own GitHub account; do not replace the instructor's starter repository. From the extracted project folder, run:

```
git init
git add .
git commit -m "Complete Employee Management API"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/Employee-API-Final.git
git push -u origin main
```

Replace the example remote URL with the real URL of the new repository. After pushing successfully, submit that repository URL along with this PDF. The local API address `http://127.0.0.1:8000/docs` is not a public GitHub URL.